

类型

Julia基础语法

苏州同元软控信息技术有限公司版权所有，
未经授权，不得复制、传播或以其他方式使用；
侵权必究。

苏州同元软控信息技术有限公司

2024年2月27日

目录

1. 类型概述、声明及特点

2. Julia中的类型

苏州同元软控信息技术有限公司
版权所有，侵权必究。

1. 类型概述、声明及特点

Julia类型系统的三个要点：动态的(dynamic)、记名的(nominative)和参数化的(parametric)。

动态的：变量的类型可变，即如果不给定类型标注只有当程序运行时才能知道变量类型。

记名的：每个类型都有名称，并且即使两个类型的含义和结构都相同，只要名称不同即为两个不同的类型。

参数化的：相当于一种对数据结构的泛化定义。

注意：

- [编译期类型]是没有任何意义的概念——变量所具有的唯一类型是程序运行时的实际类型；
- 值有类型，变量没有类型——变量仅仅是绑定给值的名称；

1. 类型概述、声明及特点

类型声明

:: 运算符：可以用来在程序中给表达式和变量附加类型注释。

作用：1. 作为断言，帮助程序确认能否正常进行；

2. 在 Julia 没办法推断类型的时候，手动提供辅助类型信息来避免性能下降

```
julia> function sinc(x)::Float64 #结果的数值类型为Float64
    if x==0
        return 1
    end
    sin(x)/(x)
end
sinc (generic function with 1 method)

julia> sinc(0)
1.0

julia> sinc(1.0) #结果的数值类型为Float64
0.8414709848078965

julia> sinc(1+2im) #结果的数值类型无法转化为Float64
ERROR: InexactError: Float64(1.416996119211876 -
0.8743911970021462im)
```

说明：

- **::** 运算符放在表示值的表达式之后时读作“前者是后者的实例”，它用来断言左侧表达式是否为右侧表达式的实例
- 如果类型断言非真，抛出一个异常，否则返回左侧的值

1. 类型概述、声明及特点

```
julia> function foo() #同foo()::Int8
    x::Int8=100 #x的数值类型为Int8
    x
end
foo (generic function with 1 method)

julia> foo()
100

julia> typeof(ans)
Int8
```

```
julia> function f(x::Int)::Float64 #x为Int, 输出为Float64
    x/10
end
f (generic function with 1 method)

julia> f(10)
1.0
```

注意：

- 当`::`操作符附加到赋值左侧的变量时，它声明的变量始终具有指定类型，如同静态类型语言的类型声明
- `::`操作符可同时用于参数类型和返回值类型，此时参数类型及返回值类型被固定

1. 类型概述、声明及特点

特点: 多重分派

分派就是指根据变量的类型选择相应的方法，单分派指的是指根据第一个参数类型去选择方法，所谓多重分派为根据参数的类型实现不同的行为。

```
julia> function sum1(a,b) #未指定a和b的数值类型
    a+b
end
sum1 (generic function with 1 method)
```

```
julia> sum1(1,2)
3
```

```
julia> sum1(1.2,2.3)
3.5
```

```
julia> sum1(1.2,4)
5.2
```

```
julia> function sum1(a::Int,b::Int) #a和b均为Int
    a-b
end
sum1 (generic function with 2 method)
```

注意：对参数类型约束后，sum1函数仍是多态的，之前的调用仍适合。

```
julia> sum1(1+2im,1) #调用方法1
2 + 2im
```

```
julia> sum1(2,3) #调用方法2
-1
```

注意：对于sum1的两种衍生方法，系统会根据给予的参数类型进行选择具体使用的衍生方法。

目录

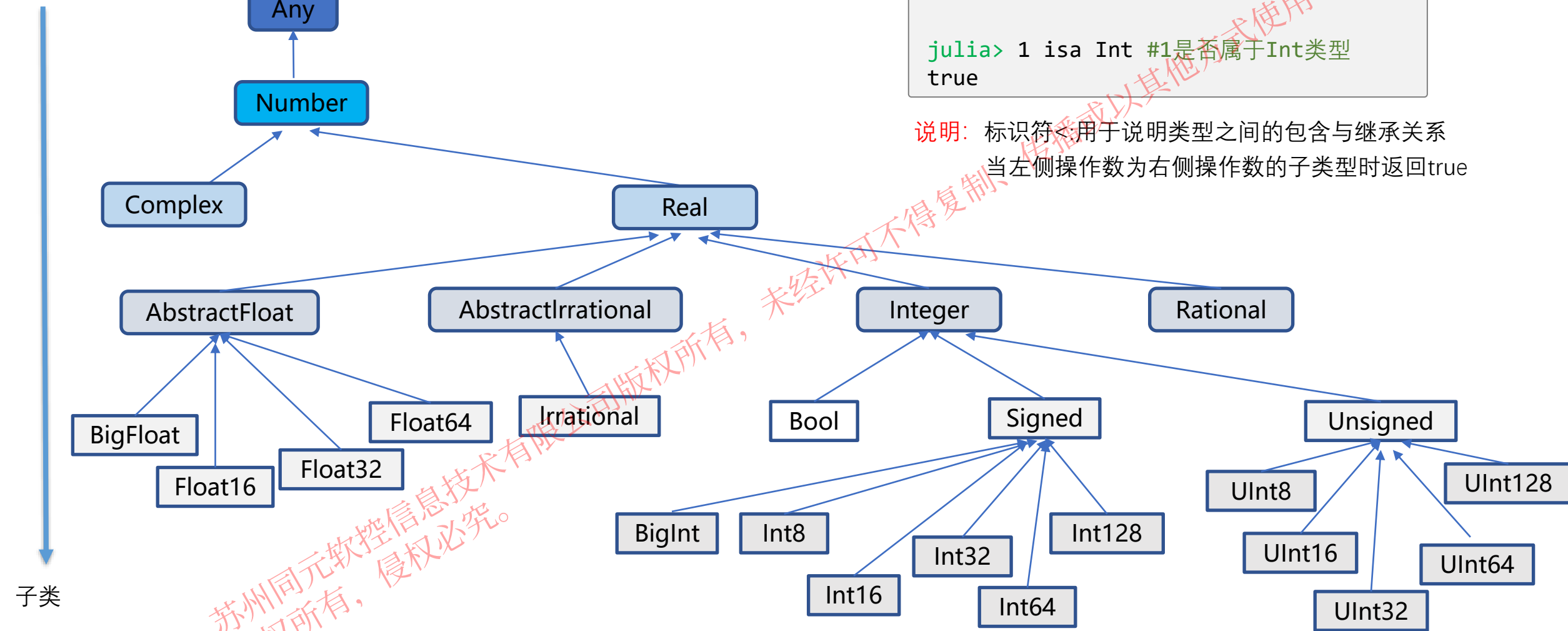
1. 类型概述、声明及特点

2. Julia中的类型

苏州同元软控信息技术有限公司
版权所有，侵权必究。

2. Julia中的类型

父类(超类型)



```
julia> Int64 <: Signed <: Integer
true

julia> 1 isa Int #1是否属于Int类型
true
```

说明: 标识符<:用于说明类型之间的包含与继承关系
当左侧操作数为右侧操作数的子类型时返回true

数值类型层次结构图

苏州同元软控信息技术有限公司
版权所有，侵权必究。



2. Julia中的类型

抽象类型：不能被实例化，只能作为类型图中的节点使用，从而描述由相关具体类型组成的集合，为具体类型提供默认实现

语法: `abstract type <<name>> end`
`abstract type <<name>> <: <<supertype>> end`

```
abstract type Number end
abstract type Real <: Number end
abstract type AbstractFloat <: Real end
abstract type Integer <: Real end
abstract type Signed <: Integer end
abstract type Unsigned <: Integer end
```

```
julia> isabstracttype{Real} #Real为抽象类型
true
```

```
julia> isabstracttype{Bool} #Bool为具体类型
false
```

```
julia> isconcretetype{Real}
false
```

```
julia> isconcretetype{Bool}
true
```

```
julia> f(a::Real)=a*a
f (generic function with 1 method)

julia> f(2);f(2.0) #Int和Float均属于Real

julia> f(2+2im)
ERROR: MethodError: no method matching f(::Complex{Int64})
Closest candidates are:
  f(::Real) at REPL[52]:1
Stacktrace:
 [1] top-level scope
      @ REPL[53]:1
```

说明:

- `isabstracttype`用于判断类型是否属于抽象类型
- `isconcretetype`用于判断类型是否属于具体类型

注意:

- 具体类型不能像抽象类型被继承，即具体类型只能是类型图中的终端或树上的叶子。

2. Julia中的类型

Any类型： 唯一的顶层类型，是所有类型的直接超类型或间接超类型。

说明：

- Any类型为默认类型
- Any类型为抽象类型，本身不能被实例化，但所有值都是它的实例。

```
julia> 2.0::Any
2.0

julia> 1//2::Any
1//2

julia> 2022::Any
2022

julia> "2022"::Any
"2022"
```

Union{}类型： 是所有相关类型的子类型。

`Union{Types...}`，Types...代表任意个类型参数，多个时用逗号隔开，可以把多个类型联合成一个类型，并让后者作为前者的同一代表，因此又称联合类型。

说明：

- `Union{}`，参数为空时，是类型图最底端类型
- `Union{}`，参数为空时，任何类型都是其超类型。

```
julia> IntOrString=Union{Integer,AbstractString}
Union{AbstractString, Integer}

julia> 2022::IntOrString
2022

julia> "2022"::IntOrString
"2022"
```

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

谢谢！

苏州同元软控信息技术有限公司版权所有，
未经许可不得复制或传播或以其他方式使用；
版权所有，侵权必究。